

Lab 07: Operating Systems & File Systems

Operating Systems and Distributed Systems

This lab aims to introduce you to CPU and memory management when using Docker and to introduce the operations of reading and writing to the storage of your system.

Additionally, it challenges the student to implement a simple filesystem, which is rather complex.

The lab is structured so the student is assisted through the tasks.

Remember to read the exercise and the accompanying text thoroughly before asking for help.

Contents

Exercise 01: Fibonacci	2
Exercise 02: CPU Usage	3
Exercise 03: Memory Usage	4
Exercise 04: Disk Read Performance	6
Fio	6
Exercise 05: Disk Write Performance	8
Exercise 06: Implement File System [Optional]	10
Code Explanation	10

Exercise 01: Fibonacci

In this exercise, the student will make a Python script that calculates the 35th Fibonacci number and prints the time it took the computer to calculate it. Below is the pseudo-code for printing out the time it takes to calculate Fib35.

```
1 StartTime = CurrentTime;  
2 Fib(35);  
3 EndTime = CurrentTime;  
4 print(EndTime - StartTime);
```

Your task

1. Make a script that calculates Fib35. *Inspiration*
2. Make the script print the calculation time. If the calculation is too fast, increase the Fib number, and if the calculation is taking too long, decrease the Fib number.

The exercise will return the following message:

```
1 <TIME>
```

Exercise 02: CPU Usage

In this exercise, the student will run the Python script inside a container and change its CPU usage.

Information 1

When starting a container with docker, it is possible to specify how much of the available CPU resources a container can consume using the `--cpus=<VALUE>` option. For example, assuming that the host machine has four cores, it is possible allocate two and a half of the cpus, using `--cpus="2.5"`.

Information 2

Additional information about resource constraints using docker can be found at https://docs.docker.com/config/containers/resource_constraints/

Your task

1. Create a Dockerfile which runs the script from the previous exercise.
2. Create a Docker image from the Dockerfile.
3. Spin up containers from the image and use different options for CPU. Notice the difference in the time it takes for the program to calculate.
4. Talk with the other group members and explain what is happening.

The exercise will return the following message:

1 <TIME>

Exercise 03: Memory Usage

In this exercise, the student will run the Python script inside a container and change its memory usage.

Information 3

When starting a container with docker, it is possible to specify the amount of memory the container can consume using the `-m=<VALUE>` option. For example, assuming that the host machine has four GBs of memory, it is possible allocate two and a half of the GB, using `-m="2.5g"`.

Information 4

When starting a container with docker, it is possible to specify the amount of memory the container is allowed to swap to disk using the `--memory-swap=<VALUE>` option. For example, it is possible to allocate 250 MB, using `--memory-swap="250m"`.

Hint 1

The `--memory-swap` option has to be used in conjunction with the `-m` option

Below is a code snippet of a python script which uses much memory:

```
1 import time
2
3 def use_space(loops):
4     for i in range(0, loops):
5         arr = bytearray(512000000)
6
7 start_time = int(round(time.time() * 1000))
8 use_space(2)
9 end_time = int(round(time.time() * 1000))
10 print('time: ' + str(end_time - start_time))
```

Your task

1. Create a Dockerfile which runs the script from the previous exercise.
2. Create a Docker image from the Dockerfile.
3. Spin up containers from the image and use different options for CPU. Notice the difference in the time it takes for the program to calculate.

4. Talk with the other group members and explain what is happening.

The exercise will return the following message:

1

<TIME>

Exercise 04: Disk Read Performance

In this exercise, the student will run a read performance test of their storage drives (HDD, SSD) using the tool `fiio`.

Fio

The Flexible I/O Tester, `fiio`, is a tool made for benchmarking I/O performance. When run, `fiio` will spawn several threads and do a user selected I/O action. The action is specified by the user in a job file.

`fiio` is installed through the package manager. The jobs we are interested in running are in the `resources` folder.

The jobs will read data (random and sequential) from your storage unit and print out a test report.

These changes must be done in both files:

```
1 runtime = 120
2 direct = 1
```

The runtime changes will make the tools run for 120 seconds and the direct changes avoid reading to memory.

Job files can be run with the following command `$ fio <FILENAME>`

The speed of a storage unit can be calculated by using the number of IOPS and the block size: $IOPS \cdot blocksize = speed(MiB/s)$.

For more about the output of the `fiio` command, <https://tobert.github.io/post/2014-04-17-fio-output-explained.html>

Your task

1. Run `fiio` with the job "fiio-rand-read.fio"
2. Run `fiio` with the job "fiio-seq-read.fio"
3. Examine the results and write down the average number of IOPS.
4. Calculate the read speed of the storage unit.

The exercise will return the following message:

```
1 Jobs: 1 (f=1): [R(1)] [100.0%] [r=1861MiB/s,w=0KiB/s] [r=7445,w=0 IOPS] [eta 00m:00s]
2 file1: (groupid=0, jobs=1): err= 0: pid=21: Wed Oct 30 13:26:52 2019
```

```

3 read: IOPS=8499, BW=2125MiB/s (2228MB/s)(249GiB/120001msec)
4 slat (usec): min=8, max=6699, avg=91.93, stdev=61.01
5 clat (usec): min=143, max=92916, avg=1788.82, stdev=747.25
6 lat (usec): min=277, max=93071, avg=1881.06, stdev=742.01
7 clat percentiles (usec):
8 | 1.00th=[ 766], 5.00th=[ 906], 10.00th=[ 1020], 20.00th=[ 1205],
9 | 30.00th=[ 1385], 40.00th=[ 1565], 50.00th=[ 1729], 60.00th=[ 1893],
10 | 70.00th=[ 2057], 80.00th=[ 2278], 90.00th=[ 2606], 95.00th=[ 2900],
11 | 99.00th=[ 3654], 99.50th=[ 4113], 99.90th=[ 5473], 99.95th=[ 6521],
12 | 99.99th=[10552]
13 bw ( MiB/s): min= 1203, max= 2438, per=99.91%, avg=2122.90, stdev=243.65,
    samples=239
14 iops      : min= 4815, max= 9754, avg=8491.52, stdev=974.59, samples=239
15 lat (usec) : 250=0.01%, 500=0.01%, 750=0.74%, 1000=8.29%
16 lat (msec) : 2=57.56%, 4=32.82%, 10=0.58%, 20=0.01%, 100=0.01%
17 cpu       : usr=2.76%, sys=79.78%, ctx=69600, majf=1, minf=1035
18 IO depths  : 1=0.1%, 2=0.1%, 4=0.1%, 8=0.1%, 16=100.0%, 32=0.0%, >=64=0.0%
19 submit    : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.0%, 32=0.0%, 64=0.0%, >=64=0.0%
20 complete  : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.1%, 32=0.0%, 64=0.0%, >=64=0.0%
21 issued rwt: total=1019915,0,0, short=0,0,0, dropped=0,0,0
22 latency   : target=0, window=0, percentile=100.00%, depth=16
23
24 Run status group 0 (all jobs):
25 READ: bw=2125MiB/s (2228MB/s), 2125MiB/s-2125MiB/s (2228MB/s-2228MB/s), io=249
    GiB (267GB), run=120001-120001msec
26
27 Disk stats (read/write):
28 sda: ios=2039907/21, merge=581/10, ticks=2162340/0, in_queue=2161920, util
    =100.00%

```

Exercise 05: Disk Write Performance

In this exercise the student will run a write performance test of their drive, using fio.

The setting is the same as the previous exercise. The only difference is that these jobs write instead of reading. The jobs can be found in the resources folder.

The jobs will write data (random and sequential) to your storage unit and print out a test report.

These changes have to be done in both files:

```
1 bs = 4K
2 numjob = 4 (in rand)
3 numjob = 1 (in seq)
```

Your task

1. Run fio with the job "fio-rand-write.fio"
2. Run fio with the job "fio-seq-write.fio"
3. Examine the results and write down the average number of IOPS.
4. Calculate the read speed of the storage unit.

The exercise will return the following message:

```
1 Jobs: 1 (f=1): [R(1)] [100.0%] [r=1861MiB/s,w=0KiB/s] [r=7445,w=0 IOPS] [eta 00m:00s]
2 file1: (groupid=0, jobs=1): err= 0: pid=21: Wed Oct 30 13:26:52 2019
3   read: IOPS=8499, BW=2125MiB/s (2228MB/s)(249GiB/120001msec)
4     slat (usec): min=8, max=6699, avg=91.93, stdev=61.01
5     clat (usec): min=143, max=92916, avg=1788.82, stdev=747.25
6       lat (usec): min=277, max=93071, avg=1881.06, stdev=742.01
7     clat percentiles (usec):
8       | 1.00th=[ 766],  5.00th=[ 906], 10.00th=[ 1020], 20.00th=[ 1205],
9       | 30.00th=[ 1385], 40.00th=[ 1565], 50.00th=[ 1729], 60.00th=[ 1893],
10      | 70.00th=[ 2057], 80.00th=[ 2278], 90.00th=[ 2606], 95.00th=[ 2900],
11      | 99.00th=[ 3654], 99.50th=[ 4113], 99.90th=[ 5473], 99.95th=[ 6521],
12      | 99.99th=[10552]
13   bw (  MiB/s): min= 1203, max= 2438, per=99.91%, avg=2122.90, stdev=243.65,
        samples=239
14   iops        : min= 4815, max= 9754, avg=8491.52, stdev=974.59, samples=239
15   lat (usec)   : 250=0.01%, 500=0.01%, 750=0.74%, 1000=8.29%
16   lat (msec)   : 2=57.56%, 4=32.82%, 10=0.58%, 20=0.01%, 100=0.01%
```

```

17  cpu          : usr=2.76%, sys=79.78%, ctx=69600, majf=1, minf=1035
18  IO depths    : 1=0.1%, 2=0.1%, 4=0.1%, 8=0.1%, 16=100.0%, 32=0.0%, >=64=0.0%
19    submit     : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.0%, 32=0.0%, 64=0.0%, >=64=0.0%
20    complete   : 0=0.0%, 4=100.0%, 8=0.0%, 16=0.1%, 32=0.0%, 64=0.0%, >=64=0.0%
21    issued rwt: total=1019915,0,0, short=0,0,0, dropped=0,0,0
22    latency    : target=0, window=0, percentile=100.00%, depth=16
23
24  Run status group 0 (all jobs):
25    READ: bw=2125MiB/s (2228MB/s), 2125MiB/s-2125MiB/s (2228MB/s-2228MB/s), io=249
      GiB (267GB), run=120001-120001msec
26
27  Disk stats (read/write):
28    sda: ios=2039907/21, merge=581/10, ticks=2162340/0, in_queue=2161920, util
      =100.00%

```

Exercise 06: Implement File System [Optional]

In this exercise, the student will implement a filesystem that uses MySQL for storage.

The resources for this exercise can be found in the resource folder.

This exercise includes nearly no help and relies on your ability to solve the problems.

Code Explanation

The methods `getaddr()` and `readdir()` are run when `$ ls -l` is executed.

`readdir()` returns a list of files.

`getaddr()` returns metadata about the file requested in the path.

`read()` is run when the content of a file is read.

You can use the `encoded()` method in the `read()` method to encode the file content to UTF-8.

Hint 1

Install `python3`, `python3-pip` and `python3-fuse`

Hint 2

Install `mysql.connector` and `fusepy` through pip:

```
$ pip install fusepy mysql-connector-python
```

Hint 3

Create a MYSQL database with a table, `Files`, containing the following columns:

- `fileid` (INT, `auto_increment`)
- `filename` (TEXT OR VARCHAR)
- `filesize` (INT)
- `filecontent` (TEXT OR VARCHAR)

Hint 4

`$ ls -l` will list all files in the directory and `$ cat *` will read the content of files in the directory.

Hint 5

If the solution is run in a Docker Container, use `--privileged`

Hint 6

TODO comments in the code show where you are supposed to include code.

Hint 7

Information about using the Python MySQL connector can be found here:

https://www.w3schools.com/python/python_mysql_getstarted.asp

Your task

1. Make the file system work.